Tenderly

Agentic Tender-to-Proposal Automation Platform

Project Specification – Frogtoberfest

Contents

1 Overview and Scope Decisions **4**

2 Problem **4**

3 Proposed Solution **4**

4 Alignment with Frogtoberfest Guidelines **5**

5 Features **5**

5.1 Must-have (MVP) . . . 5

5.2 Should-have (if ahead of schedule) . . . 5

5.3 Explicitly not building . . . 6

6 MVP Scope **6**

7 Tech Stack **6**

8 System Architecture **7**

8.1 Agent contracts . . . 7

8.2 Design rules . . . 7

9 Frontend (Next.js + TypeScript) **8**

9.1 Pages . . . 8

9.2 Structure and practices . . . 8

10 Backend **8**

10.1 Repository layout . . . 8

10.2 Core endpoints . . . 9

10.3 Pipeline orchestration . . . 9

10.4 Retrieval . . . 9

10.5 Failure handling . . . 9

11 Database **9**

12 Two-Week Development Plan **10**

12.1 Roles . . . 10

12.2 Day-by-day plan . . . 10

12.3 Working agreements . . . 11

13 Testing **11**

14 Deployment **12**

15 Documentation and Submission Checklist **12**
15.1 AI usage disclosure (to finalize with real paths) 12

16 Final Pitch Preparation **13**
16.1 Pitch structure (about 5 minutes) 13
16.2 Demo script rules 13

17 Risks and Mitigations **13**

18 Limitations and Future Improvements **13**
18.1 Known limitations 13
18.2 Future work 14

1 Overview and Scope Decisions

Tenderly is a platform that scouts public tenders, matches them to a company profile, checks eligibility, drafts a tailored proposal from the company's knowledge base, reviews it, and produces a submission-ready package after human approval.

The original idea covers discovery, matching, analysis, proposal generation, and submission. With two weeks and three people, the scope is set as follows.

- **In scope:** discovery (one live source plus a seeded dataset), matching, compliance checking, proposal generation, automated review, human approval, package export.
- **Out of scope:** automated portal submission (logins, CAPTCHAs, legal exposure), multi-tenant billing, OCR of scanned documents, multi-portal support.
- **Hard constraint:** every model call, including embeddings, uses open-weight models served locally or through an open-weight inference provider. No proprietary hosted APIs anywhere.

The demo claim is: *"From a raw tender notice to a compliance-checked, reviewed proposal package in minutes, with a human approving every critical step."*

2 Problem

SMEs lose tenders in four ways:

1. **Discovery:** notices are scattered across portals and updated daily; manual monitoring is slow and inconsistent.
2. **Triage:** deciding whether a tender fits scope, budget, location, and eligibility requires reading long documents.
3. **Compliance:** a single missing certificate, expired document, or missed clause disqualifies a bid.
4. **Proposal effort:** proposals reuse the same company information in slightly different forms, which is repetitive and error-prone under deadline pressure.

3 Proposed Solution

A pipeline of specialized AI stages, each producing typed, structured output that the next stage (code or AI) consumes:

Scout → Match → Compliance → Proposal → Review → Human approval → Package

Human approval is mandatory before a package is marked ready. Tenders that fail hard compliance checks never reach proposal generation, so the AI output directly controls what the system does next.

4 Alignment with Frogtoberfest Guidelines

Requirement	How Tenderly meets it
Qualifies as Build with AI	Every stage emits JSON validated by Pydantic; code routes, filters, gates, and aggregates on it (structured extraction, scoring/routing, RAG, multi-step chaining).
Open-source AI only	Ollama serving Qwen2.5-7B-Instruct (fallback: Llama 3.1 8B); embeddings via `bge-m3` or `nomic-embed-text` through Ollama. No paid APIs.
Core test (remove AI, product dies)	Without the AI, tenders are not parsed, ranked, checked, or drafted. The platform is only a database of raw notices.
Public GitHub repository	Monorepo with Docker Compose for one-command setup, MIT license.
Documentation	README, architecture overview, model and dependency list, limitations and future work (Section 15).
Working demo	Local Docker Compose demo on seeded tenders plus one live scrape.
Demo video / presentation	3-minute video and a short deck (Section 16).
AI usage disclosure	Table naming exact files and functions where AI output is consumed programmatically (Section 15.1).

5 Features

5.1 Must-have (MVP)

- Company profile: sectors, services, location, certifications, annual turnover, past projects.
- Knowledge base: upload company documents (PDF/DOCX/TXT); chunked and embedded.
- Tender ingestion: one live portal scraper plus a seeded dataset of 30–40 tenders.
- Scout: extract title, buyer, deadline, budget, location, scope, requirements, and required documents into structured JSON.
- Matching: relevance score (0–100) with short reasons; ranked dashboard.
- Compliance: per-requirement pass/fail/unclear with evidence from the knowledge base; deadline and document-expiry checks.
- Proposal generation: section-by-section drafts grounded in retrieved knowledge base chunks.
- Review: scores completeness, relevance, and compliance; lists gaps.
- Human approval gate and export of a proposal (DOCX/PDF) plus a checklist.
- Audit log of every agent run (inputs, outputs, model, latency).

5.2 Should-have (if ahead of schedule)

- One automatic revision pass: Review gaps are fed back to Proposal.
- Scheduled scouting (every N hours).
- Side-by-side view of the tender requirement and the proposal text that addresses it.

5.3 Explicitly not building

Automated portal submission, user accounts beyond a single demo company, notifications, payments, fine-tuning.

6 MVP Scope

The MVP is done when the following demo path works end to end without manual intervention except approvals:

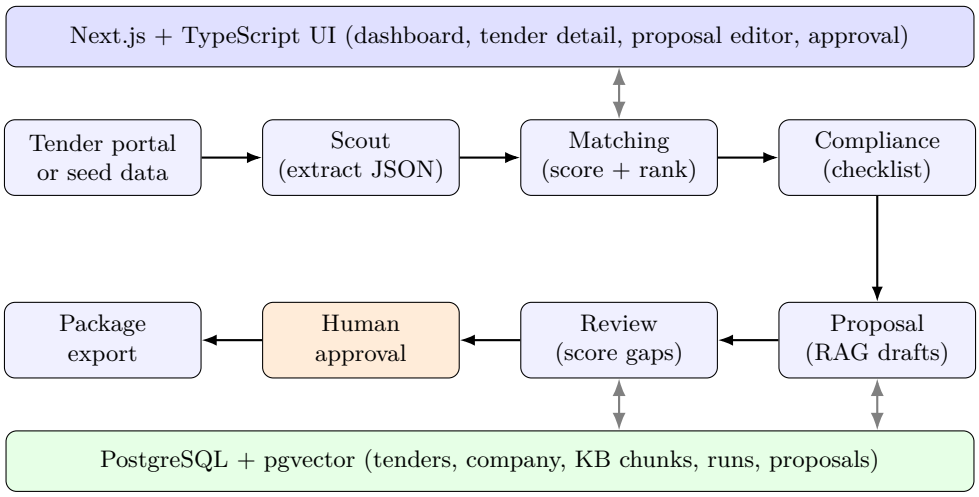
1. User creates a company profile and uploads 3–5 company documents.
2. System ingests tenders (live or seeded) and extracts structured data.
3. Dashboard shows tenders ranked by match score with reasons.
4. User opens a top tender; the compliance matrix shows pass/fail/unclear per requirement with evidence.
5. User triggers proposal generation; the draft appears section by section.
6. Review Agent scores the draft and lists gaps.
7. User edits if needed, approves, and downloads the submission package.

Cut line. If the team is behind at the Day 7 checkpoint, drop in this order: scheduled scouting, automatic revision pass, DOCX export (keep PDF or Markdown), live scraper (keep seeded data).

7 Tech Stack

Layer	Choice	Reason
Frontend	Next.js (App Router), TypeScript, Tailwind CSS	Required; fast UI iteration.
Backend API	Python 3.11, FastAPI, Pydantic	Best ecosystem for AI pipelines; schema validation built in.
Orchestration	Plain Python pipeline with a state machine	Fewer moving parts than agent frameworks; easy to debug and explain.
LLM serving	Ollama	Simple local serving, JSON-schema constrained output.
LLM	Qwen2.5-7B-Instruct (fallback Llama 3.1 8B)	Open weights, good structured output at 7B.
Embeddings	`bge-m3` or `nomic-embed-text` via Ollama	Open; `bge-m3` is multilingual if tenders mix English and Nepali.
Database	PostgreSQL with pgvector	One store for relational data and vector search.
Scraping	httpx, BeautifulSoup, Playwright (only if needed)	Lightweight first.
Document parsing	PyMuPDF, python-docx	Text extraction; no OCR in MVP.
Export	python-docx, WeasyPrint or ReportLab	Proposal DOCX/PDF.
Scheduling	APScheduler	No separate queue infrastructure.
Testing	pytest, Vitest, Playwright (one E2E test)	Minimal but meaningful coverage.
Deployment	Docker Compose	One-command run for judges.

8 System Architecture



All stages run in the FastAPI backend and call Ollama over HTTP. Each stage reads from and writes to PostgreSQL, so runs are resumable and auditable. The frontend talks only to the FastAPI REST API.

8.1 Agent contracts

Stage	Input	Structured output	Consumed by
Scout	Raw notice text/HTML/PDF text	`TenderRecord`: deadline, budget, location, scope, requirements[], required_docs[]	DB write; deadline and budget filters
Matching	`TenderRecord` + company profile + top-k KB chunks	`MatchResult`: score, reasons[], decision (pursue/skip)	Ranking; threshold decides whether Compliance runs
Compliance	Requirements + company profile + documents with expiry dates	`ComplianceReport`: per-item status, evidence, severity	Hard failures block proposal generation
Proposal	Tender + compliance report + retrieved KB chunks per section	`ProposalSection[]` with source chunk ids	Assembler builds the document
Review	Proposal + requirements	`ReviewReport`: scores, gaps[], verdict	Verdict sets status; gaps feed optional revision

8.2 Design rules

- Every LLM call uses JSON-schema constrained output and Pydantic validation; invalid output triggers one retry, then the stage is marked failed.
- Temperature 0 for extraction, matching, compliance, and review; low-moderate for proposal text.
- Deadline, budget, and document-expiry checks are deterministic code, not LLM judgment. The LLM only extracts the values.

- Proposal text must cite the KB chunk ids it used; sections without grounding are flagged by Review.
- No stage submits anything externally.

9 Frontend (Next.js + TypeScript)

9.1 Pages

Route	Purpose
`/`	Dashboard: ranked tenders, score, deadline countdown, status badge.
`/company`	Profile form and knowledge base upload with ingestion status.
`/tenders/[id]`	Extracted fields, match reasons, compliance matrix with evidence.
`/tenders/[id]/proposal`	Section-by-section editor, review report, source citations, approve button.
`/runs`	Pipeline run log: stage, status, duration, model.

9.2 Structure and practices

- App Router with server components for list pages and client components for editor and live status.
- A typed API client generated from the FastAPI OpenAPI schema (`openapi-typescript`) so frontend types match backend schemas.
- Progress via polling every 2 seconds on a run endpoint; switch to SSE only if time remains.
- Compliance matrix is a table with green/red/amber rows and an expandable evidence cell; this is the strongest demo visual.
- Approval button is disabled until Review verdict is not "fail" and the user has opened the proposal.
- Component library: shadcn/ui plus Tailwind to avoid design time.

10 Backend

10.1 Repository layout

```
Tenderly/
  web/ # Next.js + TypeScript
  api/
    main.py
    routers/ # tenders.py, company.py, proposals.py, runs.py
    agents/ # scout.py, matching.py, compliance.py, proposal.py, review.py
    pipeline/ # orchestrator.py (state machine), llm.py (Ollama client)
    schemas/ # Pydantic models shared by all stages
    ingest/ # scraper.py, parse_docs.py, embed.py
    export/ # docx.py, pdf.py, package.py
    tests/
  docker-compose.yml
  README.md ARCHITECTURE.md AI_DISCLOSURE.md LICENSE
```

10.2 Core endpoints

Endpoint	Function
`POST /company`	Create or update company profile.
`POST /company/documents`	Upload document; parse, chunk, embed.
`POST /tenders/scrape`	Run scraper or load seed data; enqueue Scout.
`GET /tenders`	Ranked list with filters.
`GET /tenders/{id}`	Tender detail, match result, compliance report.
`POST /tenders/{id}/analyze`	Run Matching and Compliance.
`POST /tenders/{id}/proposal`	Run Proposal and Review.
`POST /proposals/{id}/approve`	Human approval; sets status.
`GET /proposals/{id}/package`	Download submission package.
`GET /runs/{id}`	Run status for polling.

10.3 Pipeline orchestration

The orchestrator is a small state machine over the tender status: `new` → `extracted` → `matched` → `checked` → `drafted` → `reviewed` → `approved`. Each transition writes an `agent_runs` row with input hash, output JSON, model name, and latency. Branches:

- Match score below threshold ⇒ status `skipped`; Compliance does not run.
- Any hard compliance failure ⇒ status `blocked`; the proposal is not generated unless the user overrides.
- Review verdict "fail" ⇒ one revision pass (if enabled), then back to the user.

10.4 Retrieval

KB documents are split into 400–600 token chunks with overlap, embedded, and stored in pgvector. Matching and Proposal retrieve top-k chunks by cosine similarity. For Proposal, retrieval runs per section (for example, "relevant experience", "technical approach") using the section topic plus the tender requirement as the query.

10.5 Failure handling

One retry on schema-invalid output; a timeout per call; a clear failed state in the UI; the demo dataset is pre-analyzed and cached so a model failure on stage does not kill the pitch.

11 Database

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE company (
  id SERIAL PRIMARY KEY,
  name TEXT, sectors TEXT[], services TEXT, location TEXT,
  turnover_npr BIGINT, certifications TEXT[], profile_json JSONB
);

CREATE TABLE documents (
  id SERIAL PRIMARY KEY, company_id INT REFERENCES company(id),
  filename TEXT, doc_type TEXT, expires_on DATE, raw_text TEXT
);
```

```
CREATE TABLE kb_chunks (
  id SERIAL PRIMARY KEY, document_id INT REFERENCES documents(id),
  content TEXT, embedding VECTOR(1024) -- match the embedding model dimension
);

CREATE TABLE tenders (
  id SERIAL PRIMARY KEY, source_url TEXT UNIQUE, raw_text TEXT,
  title TEXT, buyer TEXT, deadline TIMESTAMP, budget BIGINT,
  location TEXT, record_json JSONB, -- TenderRecord
  status TEXT DEFAULT 'new'
);

CREATE TABLE matches (
  tender_id INT REFERENCES tenders(id), score INT,
  reasons JSONB, decision TEXT
);

CREATE TABLE compliance_items (
  id SERIAL PRIMARY KEY, tender_id INT REFERENCES tenders(id),
  requirement TEXT, status TEXT, severity TEXT, evidence TEXT
);

CREATE TABLE proposals (
  id SERIAL PRIMARY KEY, tender_id INT REFERENCES tenders(id),
  version INT, sections JSONB, review_json JSONB,
  status TEXT DEFAULT 'draft', approved_at TIMESTAMP
);

CREATE TABLE agent_runs (
  id SERIAL PRIMARY KEY, tender_id INT, stage TEXT, model TEXT,
  input_hash TEXT, output_json JSONB, latency_ms INT,
  ok BOOLEAN, created_at TIMESTAMP DEFAULT now()
);
```

Embedding dimension must match the chosen model (1024 for `bge-m3`, 768 for `nomic-embed-text`); fix this on Day 1 and do not change it.

12 Two-Week Development Plan

12.1 Roles

Member A
: AI pipeline lead: model setup, schemas, prompts, all five agents, evaluation set.

Member B
: Backend and data lead: FastAPI, database, scraper, ingestion, orchestrator, export, Docker.

Member C
: Frontend and product lead: Next.js UI, UX flow, README and documentation, demo video, pitch.

Shared schemas (`TenderRecord`, `MatchResult`, `ComplianceReport`, `ProposalSection`, `ReviewReport`) are agreed on Day 1 and treated as the contract; changes need all three members' agreement. This lets frontend and backend work in parallel using mocked responses.

12.2 Day-by-day plan

Day	A: AI pipeline	B: Backend/data	C: Frontend/product
1	Install Ollama, pull model and embedder, test JSON-schema output; draft Pydantic schemas	Repo, Docker Compose (api, db, ollama), FastAPI skeleton, migrations	Next.js + TS scaffold, Tailwind/shadcn, API client from OpenAPI, wireframes
2	Scout extraction prompt v1 on 10 real notices	Scraper for one portal; store raw text; build seed set of 30–40 tenders	Company profile form, KB upload UI
3	Measure Scout field accuracy; fix prompt	PDF/DOCX parsing, chunking, embedding, pgvector search	Tender list page with mock data
4	Matching agent: retrieval + LLM score JSON	Match endpoints, persist results, ranking query	Dashboard with scores, reasons, deadline countdown
5	Compliance agent: requirement checklist with evidence	Document expiry and deadline/budget rules in code; compliance endpoints	Tender detail page with compliance matrix
6	Label 15 tenders; tune Matching and Compliance	Orchestrator state machine, run logging, retries	Runs page, polling for status
7	**Checkpoint M1:** Seed tender → extracted → matched → checked, visible in the UI. Apply cut line if behind.		
8	Proposal agent: section plan, per-section RAG	Proposal storage, versioning, endpoints	Proposal editor view
9	Citations to KB chunks, prompt tuning	DOCX/PDF export, submission package ZIP	Approval flow and package download
10	Review agent: scores, gaps, verdict; optional revision pass	Approval gating, audit log	Review panel, gap list, source citations
11	**Checkpoint M2 / feature freeze:** full flow end to end. Bug bash; no new features after today.		
12	Run evaluation set; fix worst failure modes; test fallback model	Unit and integration tests; one-command setup; pre-cache demo analyses	README, ARCHITECTURE.md, AI_DISCLOSURE.md drafts
13	Final tuning of demo tenders	Clean-machine install test; fix deployment issues	Record demo video; finish limitations section
14	Pitch rehearsal (twice), submission checklist, buffer for fixes. No new code unless a demo blocker.		

12.3 Working agreements

- Short daily sync (10 minutes): blocked on what, done what, next what.
- Merge to `main` daily through small pull requests; each member reviews one other member's PR per day.
- Day 7 and Day 11 are hard checkpoints; scope is cut, not the quality of what remains.

13 Testing

- **Deterministic unit tests (pytest):** deadline parsing, budget comparison, document expiry, state machine transitions, package builder.
- **Schema tests:** every agent output validates against its Pydantic model, including a deliberately malformed output to confirm the retry-then-fail path.
- **Golden set (15 tenders, hand-labeled):** expected deadline, budget, requirement list, and a relevant/not-relevant label for the demo company.
- **Metrics tracked:**
  - Scout: field-level accuracy for deadline, budget, and required documents.
  - Matching: whether labeled-relevant tenders appear in the top 3.
  - Compliance: recall on hand-labeled mandatory requirements (missing a hard requirement is the worst error).

  - Proposal: percentage of sections with at least one valid KB citation.

- **End-to-end test (Playwright):** one scripted run of the demo path on seeded data.
- **Manual demo rehearsal:** run the full demo path on a clean checkout at least twice before recording.

Report the golden-set numbers honestly in the README. Weak numbers with a clear explanation score better than unmeasured claims.

14 Deployment

- **Primary:** `docker compose up` starts web, api, postgres (pgvector), and ollama; a setup script pulls the models. Judges and teammates run the identical stack.
- **Model endpoint:** the LLM client reads `LLM_BASE_URL` and `LLM_MODEL` from the environment. This allows swapping to a hosted open-weight inference provider if no GPU is available. The endpoint must serve open-weight models only, and this is stated in the README.
- **Hardware reality check:** a 7B model on CPU is slow. Pre-cache demo analyses so the live demo shows real outputs instantly and runs one fresh tender live if latency allows.
- **Fallback:** a recorded video of the full flow is the guaranteed backup if the live demo fails.
- **Optional:** frontend on Vercel pointing to a backend run on a team laptop; not required and not worth the debugging time unless everything else is done.

15 Documentation and Submission Checklist

Public GitHub repository with an open-source license.

README: problem, solution, quick start, model and dependency list, demo instructions.

ARCHITECTURE.md: this document's architecture section, plus the agent contracts.

Limitations and future improvements: no automated submission, one portal, no OCR, small-model errors, English-first, single-tenant.

Working demo reproducible from a clean machine.

Demo video (about 3 minutes) and presentation deck.

AI usage disclosure (below).

15.1 AI usage disclosure (to finalize with real paths)

File / function	Where the output is consumed programmatically
`agents/scout.py::extract_tender`	Parsed into `TenderRecord`; deadline and budget written to DB columns and used by filters.
`agents/matching.py::score_tender`	Score and decision determine ranking and whether Compliance runs.
`agents/compliance.py::check_requirements`	Hard requirements stored; any hard failure sets status `blocked`.
`agents/proposal.py::generate_section`	Section text with cited chunk ids assembled by `export/package.py`.
`agents/review.py::review_proposal`	Verdicts gate proposal status; gaps trigger the revision pass.
`ingest/embed.py::embed_chunks`	Embeddings used for retrieval in Matching and Proposal.

16 Final Pitch Preparation

16.1 Pitch structure (about 5 minutes)

1. **Problem (45 s):** one concrete story of an SME missing or losing a tender on a technicality.
2. **Solution (30 s):** the pipeline diagram.
3. **Live demo (2.5 min):** dashboard → compliance matrix with a failing requirement → proposal generation → review gaps → approval and package.
4. **How AI is used (45 s):** point to structured outputs driving routing, blocking, and gating. State that everything runs on open-weight models.
5. **Results and limits (30 s):** golden-set numbers, what is not built, what comes next.

16.2 Demo script rules

- Use one prepared company profile and three prepared tenders: one strong match, one that is blocked by compliance, one skipped as irrelevant. This shows the AI changing system behavior, which is exactly what the guidelines test.
- Show the audit log once to prove outputs are consumed by code.
- Rehearse at least twice; assign who talks and who drives the screen.
- Prepare answers for the likely questions: why not auto-submit, how accurate is a 7B model, what happens on a hallucinated requirement, how is the company's data protected.

17 Risks and Mitigations

Risk	Likelihood	Mitigation
Portal scraper breaks or blocks	High	Seeded dataset from Day 2; scraper is a bonus, not a dependency.
7B model extracts wrong deadline or budget	Medium	Deterministic validation; show extracted value next to source text; golden-set tuning.
Hallucinated compliance evidence	Medium	Evidence must cite a KB chunk id; items without evidence become "unclear", never "pass".
Slow inference on demo hardware	High	Pre-cached analyses; smaller quantized model; hosted open-weight fallback.
Scope creep	High	Cut line and Day 11 feature freeze.
Schema drift between frontend and backend	Medium	Generated TypeScript client from OpenAPI; shared schema agreed Day 1.
Proprietary API sneaks in (e.g., a default embedding)	Low	Audit all dependencies on Day 12; single `llm.py` client for all calls.

18 Limitations and Future Improvements

18.1 Known limitations

- No automated submission; the final upload is manual.

- One live portal; layout changes break the scraper.
- No OCR; scanned documents are not read.
- Small open models can misread complex clauses; the human approval gate exists for this reason.
- Single company, no authentication.

18.2 Future work

- Browser-driven assisted submission with a human confirmation step at every irreversible action.
- More portals through a plug-in scraper interface.
- OCR and layout-aware parsing for scanned tenders.
- Fine-tuned extraction model on labeled tenders.
- Multi-user workspaces, deadline alerts, and win/loss feedback to improve matching.